8086 Logical Instructions

1. Overview

Concept

Logical instructions perform bitwise operations on data. They are used for masking bits, testing conditions, clearing registers, and preparing data for decision making.

Rules / Notes

Key Properties

- Operate on **8-bit or 16-bit** operands
- Memory-to-memory operations are **not allowed**
- Mostly affect **ZF, SF, PF**
- CF and OF are usually cleared

2. Logical Instructions

Concept

Logical instructions perform **bitwise operations** on data. They are mainly used for **masking bits, setting flags, clearing registers, and testing conditions**.

2.1 AND Instruction

Concept

AND performs a bitwise AND operation. A result bit is **1 only if both corresponding bits are 1**; otherwise, the result bit is **0**.

Rules / Notes

Syntax

```
AND destination, source
```

Operands

- Destination: register or memory
- Source: register, memory, or immediate
- Memory-to-memory operation is **not allowed**

Flags

- ZF, SF, PF are updated
- CF = 0, OF = 0

Example

Bitwise Example

$$\begin{array}{r} 0101 \\ \wedge\ 0011 \\ \hline 0001 \end{array}$$

Example

```
MOV AL, 05H      ; 0101
AND AL, 03H      ; 0011
; AL = 01H
```

Exam Tip

AND is commonly used for **masking bits**. Example: `AND AL, 0FH` clears the upper nibble.

2.2 OR Instruction

Concept

OR performs a bitwise OR operation. A result bit is **1 if either or both corresponding bits are 1**; it is **0 only if both bits are 0**.

Rules / Notes

Syntax

```
OR destination, source
```

Operands

- Destination: register or memory
- Source: register, memory, or immediate
- Memory-to-memory operation is **not allowed**

Flags

- ZF, SF, PF are updated
- CF = 0, OF = 0

Example

Bitwise Example

$$\begin{array}{r} 0101 \\ \vee\ 0011 \\ \hline 0111 \end{array}$$

Example

```
MOV AL, 05H
OR  AL, 03H
; AL = 07H
```

Exam Tip

OR is used to **set specific bits** without affecting others.

2.3 XOR Instruction

Concept

XOR performs a bitwise exclusive-OR operation. A result bit is **1 if bits are different**; it is **0 if bits are the same**.

Rules / Notes

Syntax

```
XOR destination, source
```

Operands

- Destination: register or memory
- Source: register, memory, or immediate
- Memory-to-memory operation is **not allowed**

Flags

- ZF, SF, PF are updated
- CF = 0, OF = 0

Example

Bitwise Example

$$\begin{array}{r} 0101 \\ \oplus\ 0011 \\ \hline 0110 \end{array}$$

Example

```
MOV AL, 05H
XOR AL, 03H
; AL = 06H
```

Example

```
XOR AX, AX      ; AX = 0000H
```

Explanation: XORing a register with itself always produces zero.

Exam Tip

Fastest way to clear a register: `XOR reg, reg`

2.4 NOT Instruction

Concept

NOT performs a bitwise one's complement operation. Each bit of the operand is inverted:

$$1 \to 0, \quad 0 \to 1$$

Rules / Notes

Syntax

```
NOT destination
```

Operands

- Destination: register or memory

Flags: NOT does **not affect any flags**.

Example

Bitwise Example

$$\begin{array}{r} 0101\ 0011 \\ \text{NOT} \\ \hline 1010\ 1100 \end{array}$$

Example

```
MOV AL, 53H
NOT AL
; AL = ACH
```

Exam Tip

NOT is commonly used for **bit inversion and complement masks**.

3. TEST Instruction

Concept

TEST performs AND operation but does **not store** the result. Used only to update flags.

Rules / Notes

Syntax

```
TEST destination, source
```

Flags:

- ZF, SF, PF updated
- CF = OF = 0

Example

```
TEST AL, 01H
JZ EVEN
```

Explanation: Tests whether number is even or odd.

Exam Tip

TEST is preferred over AND when the value must not be changed.

Quick Revision

- **AND:** masking bits
- **OR:** setting bits
- **XOR:** toggling / clearing
- **NOT:** bit inversion
- **TEST:** check bits without modifying data